

LAPORAN PRAKTIKUM
PEMOGRAMAN BERORIENTASI OBJEK
“MODUL 1 PRAKTIKUM”



disusun Oleh:
FILZI JELILA INDA ROBBANI
(2511533019)

Dosen Pengampu:
NURFIAH, S.ST, M.Kom

Asisten Praktikum:
DAWI SYAUQI

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Dengan memanjatkan puji syukur kehadirat Allah SWT, atas limpahan rahmat dan karunia-Nya sehingga praktikum Pemograman Berorientasi Objek kali ini dapat saya selesaikan dengan baik dan lancar. Shalawat serta salam semoga senantiasa tercurahkan kepada junjungan kita Nabi Muhammad SAW.

Laporan ini disusun untuk memenuhi salah satu tugas pada mata kuliah Praktikum Pemograman Berorientasi Objek di Universitas Andalas.

Saya menyampaikan terima kasih kepada dosen pengampu dan asisten praktikum yang telah membimbing serta memberikan arahan selama proses pembelajaran di kelas maupun di laboratorium komputer. Selain itu, saya juga berterima kasih kepada teman-teman praktikan atas bantuan dan dukungan yang diberikan, sehingga laporan ini dapat terselesaikan dengan baik.

Saya menyadari laporan ini masih jauh dari sempurna. Oleh karena itu, saya mohon maaf apabila terdapat kekurangan maupun kesalahan dalam penyusunan laporan ini. Harapan saya, semoga laporan ini dapat memberikan manfaat serta menambah pengetahuan bagi pembaca sekalian.

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB I PENDAHULUAN.....	4
1.1 Teori	4
1.2 Tujuan.....	4
1.3 Alat.....	5
BAB II PEMBAHASAN	12
2.1 Langkah Kerja Praktikum	12
2.1.1 Membuat kelas Rekening	12
2.1.2 Membuat kelas kedua (Main)	14
2.2 Tugas Praktikum	16
2.2.1 Deskripsi Tugas	16
2.2.2 Pengerjaan Latihan/Tugas No. 1	17
2.2.3 Pengerjaan Latihan/Tugas No. 2	19
BAB III PENUTUP	22
3.1 Kesimpulan	22
DAFTAR PUSTAKA.....	23

BAB I

PENDAHULUAN

1.1 Teori

Dalam Pemrograman Berorientasi Objek (PBO), perangkat lunak dimodelkan berdasarkan entitas di dunia nyata. Dua pilar paling mendasar dalam pemodelan ini adalah Class dan Object.

- Class sebagai Blueprint, Menurut Deitel & Deitel (2017) dalam bukunya Java: How to Program, sebuah class adalah cetak biru (blueprint) dari sebuah objek. Class mendefinisikan struktur data (attributes/state) dan sekumpulan operasi (methods/behavior) yang dapat dilakukan pada data tersebut. Class sendiri tidak memakan ruang memori untuk menyimpan nilai data, melainkan hanya berfungsi sebagai tipe data referensi.
- Object dan State, Objek adalah instansiasi konkret dari sebuah class yang dialokasikan di dalam memori komputer. Setiap objek memiliki state/kondisi yang independen. Sierra & Bates (2005) dalam Head First Java mengilustrasikan bahwa dua objek dari class yang sama misalnya Rekening dapat memiliki nilai state seperti saldo yang sama sekali berbeda, dan perubahan pada satu objek tidak akan memengaruhi objek lainnya.
- Constructor adalah blok kode khusus yang dieksekusi secara otomatis pada saat objek dibuat menggunakan keyword new. Fungsinya adalah untuk menginisialisasi state awal dari objek tersebut (Oracle, 2023).

Dalam konteks perbankan, Class adalah konsep "Rekening", Object adalah "Rekening milik Mawar" atau "Rekening milik Melati", State adalah "Saldo saat ini", dan Behavior adalah aktivitas seperti "Setor Tunai" yang akan memanipulasi state saldo tersebut

1.2 Tujuan

Tujuan dilakukannya praktikum ini adalah sebagai berikut:

- a. Mendefinisikan class sebagai model dari entitas dunia nyata
- b. Mendefinisikan object dan memahami alokasi memori independent
- c. Menggunakan constructor untuk inisialisasi awal state
- d. Manipulasi state (atribut) melalui behavior (method) menggunakan interaksi CLI

1.3 Alat

- a. Computer / laptop yang telah terinstall JDK dan Eclipse .

BAB II

PEMBAHASAN

2.1 Langkah Kerja Praktikum

2.1.1 Membuat kelas Rekening

- 1) Pertama buka apl Eclipse IDE for Java Developers, lalu buatlah project java baru dengan nama PrakPBO_C_nim terus pilih “New”, pilih package bikin dengan nama Pekan1 dan class. Buat nama class “Rekening.java”.

```
package Pekan1;  
  
public class Rekening {
```

- 2) Lalu, tambahkan State/Atribut.

```
public class Rekening {  
    String nomorRekening;  
    String namaPemilik;  
    double saldo;
```

- 3) Buat Constructor

```
public Rekening(String nomor, String nama, double  
saldoAwal) {  
    nomorRekening = nomor;  
    namaPemilik = nama;  
    saldo = saldoAwal;  
    NumberFormat rupiah =  
NumberFormat.getCurrencyInstance(new Locale("id",  
"ID"));  
    System.out.println("Rekening atas nama "+  
namaPemilik + " berhasil dibuat dengan saldo Rp" +  
rupiah.format(saldo));  
}
```

Constructor tsb digunakan untuk memberikan nilai awal pada rekening dengan menerima 3 parameter, yaitu nomor sebagai nomor rekening, nama sebagai nama pemilik, dan saldoAwal sebagai saldo awal. Nilai tsb disimpan ke atribut nomorRekening, namaPemilik, dan saldo.

Perintah `system.out.println()` untuk menampilkan pesan bahwa rekening berhasil dibuat beserta nama pemilik dan saldo awal yang

sudah diformat dalam rupiah. Pemformatan rupiah ini dilakukan diluar instruksi modul, melainkan instruksi tugas tambahan poin diwaktu praktikum. Pemformatan rupiah dilakukan dengan menggunakan NumberFormat yang diatur dengan Locale ID. (jangan lupa import java.text.NumberFormat; dan import java.util.Locale;)

4) Buat method/behavior

```
public void setorTunai(double nominal) {
    if (nominal > 10000 ) {
        saldo += nominal;
        System.out.println("Setor tunai Rp"+
nominal + " berhasil. Saldo saat ini : " + saldo);
    }else {
        System.out.println("Gagal : Nominal
setor harus lebih dari 0!");
    }
}
public void cekInformasi() {
    System.out.println("--- INFO REKENING ---");
    System.out.println("No. Rekening : " +
nomorRekening);
    System.out.println("Nama Pemilik : " +
namaPemilik);
    NumberFormat rupiah =
NumberFormat.getCurrencyInstance(new Locale("id", "ID"));
    System.out.println("Saldo akhir : " +
rupiah.format(saldo));
    System.out.println("-----");
}
```

Methode setorTunai(double nominal) digunakan untuk melakukan penyetoran uang ke rekening dengan menerima parameter nominal sebagai jumlah uang yang disetor.

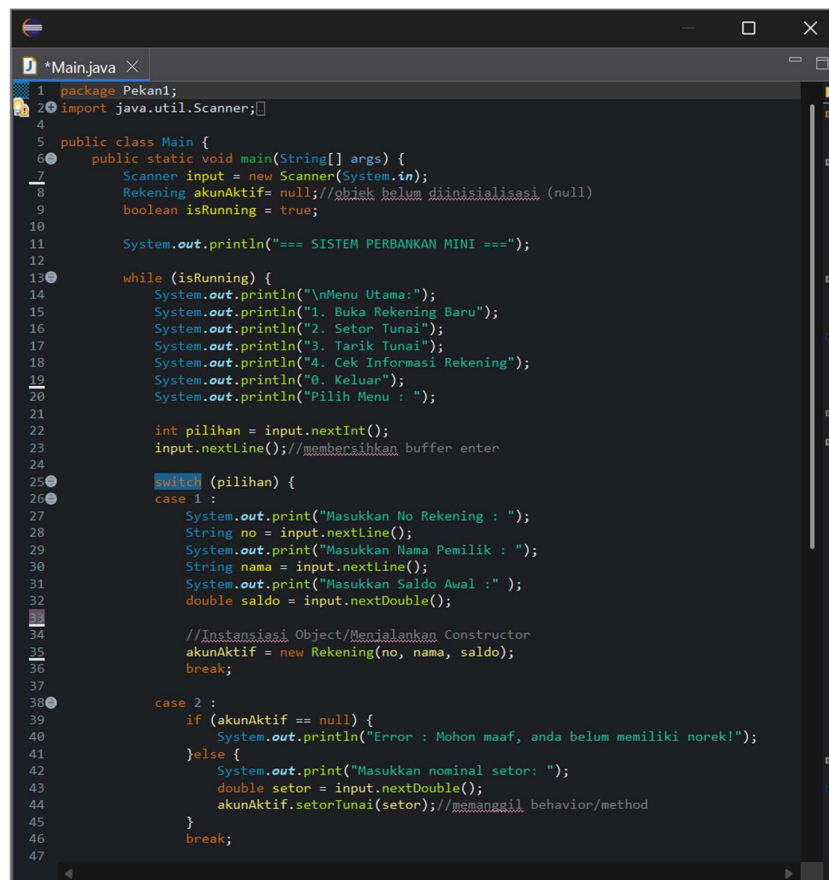
Program menggunakan kondisi if untuk nge cek apakah nominal setor lebih dari 10.000 (sebelumnya sesuai modul pengecekan nominal hanyalah harus besar dari 0, disini sudah besar dari 10.000 karena dari penugasan disela-sela praktikum), jika memenuhi maka nilai nominal akan ditambahkan ke saldo menggunakan operator += dan program menampilkan pesan bahwa setor tunai berhasil beserta saldo saat ini, sedangkan jika nominal tidak lebih dari 10.000, program akan masuk ke bagian else dan menampilkan pesan bahwa penyetoran gagal.

Lalu, method `cekInformasi()` digunakan untuk menampilkan informasi rekening berupa nomor rekening, nama pemilik, dan saldo akhir. Pada method ini, `NumberFormat` dengan `Locale("id", "ID")` digunakan agar saldo ditampilkan dalam format mata uang Rupiah, kemudian seluruh informasi rekening ditampilkan menggunakan `System.out.println()`.

Seperti pada langkah 4, pemformatan menjadi rupiah dilakukan untuk penugasan sela-sela praktikum.

2.1.2 Membuat kelas kedua (Main)

- 1) Klik kanan pada package `Pekan1` terus pilih “New”, pilih class. Buat nama class “Main.java”.
- 2) Masukkan syntax seperti berikut.



```
1 package Pekan1;
2 import java.util.Scanner;
3
4
5 public class Main {
6     public static void main(String[] args) {
7         Scanner input = new Scanner(System.in);
8         Rekening akunAktif= null; //objek belum diinisialisasi (null)
9         boolean isRunning = true;
10
11         System.out.println("=== SISTEM PERBANKAN MINI ===");
12
13         while (isRunning) {
14             System.out.println("\nMenu Utama:");
15             System.out.println("1. Buka Rekening Baru");
16             System.out.println("2. Setor Tunai");
17             System.out.println("3. Tarik Tunai");
18             System.out.println("4. Cek Informasi Rekening");
19             System.out.println("0. Keluar");
20             System.out.println("Pilih Menu : ");
21
22             int pilihan = input.nextInt();
23             input.nextLine(); //membersihkan buffer enter
24
25             switch (pilihan) {
26                 case 1 :
27                     System.out.print("Masukkan No Rekening : ");
28                     String no = input.nextLine();
29                     System.out.print("Masukkan Nama Pemilik : ");
30                     String nama = input.nextLine();
31                     System.out.print("Masukkan Saldo Awal : ");
32                     double saldo = input.nextDouble();
33
34                     //Instansiasi Object/Menjalankan Constructor
35                     akunAktif = new Rekening(no, nama, saldo);
36                     break;
37
38                 case 2 :
39                     if (akunAktif == null) {
40                         System.out.println("Error : Mohon maaf, anda belum memiliki norek!");
41                     } else {
42                         System.out.print("Masukkan nominal setor: ");
43                         double setor = input.nextDouble();
44                         akunAktif.setorTunai(setor); //memanggil behavior/method
45                     }
46                     break;
47             }
48         }
49     }
50 }
```

Program kedua Main.java digunakan sebagai class utama yang mengelola keseluruhan struktur class Rekening.java . Pada awal program, digunakan Scanner untuk menerima input dari keyboard user, kemudian dibuat variabel akunAktif bertipe Rekening yang awalnya bernilai null, artinya belum ada rekening yang sedang digunakan. Variabel isRunning bernilai true digunakan sebagai kondisi agar program terus berjalan menggunakan perulangan while.

Di while, program menampilkan beberapa pilihan menu yang terdiri dari buka rekening baru, setor tunai, tarik tunai, cek informasi rekening, dan keluar. Input pilihan user disimpan dalam variabel pilihan, kemudian switch digunakan untuk menentukan proses berdasarkan pilihan tsb.

Pada pilihan 1/case 1, program digunakan untuk membuka rekening baru. User diminta memasukkan nomor rekening, nama pemilik, dan saldo awal. Data tersebut kemudian digunakan untuk membuat objek baru dengan new Rekening(no, nama, saldo). Objek yang dibuat disimpan ke dalam akunAktif, sehingga rekening tersebut langsung menjadi rekening yang sedang digunakan.

Pada pilihan 2/case 2, program melakukan pengecekan dulu apakah akunAktif masih null. Jika null atau masih belum ada rekening, program menampilkan pesan error, sedangkan jika sudah ada, user diminta memasukkan nominal setor dan program memanggil method setorTunai() dari objek akunAktif.

Pada pilihan 3/case 3, pada gambar memang sudah ada, namun itu adalah jawaban untuk latihan/tugas no 1. Sebelumnya sesuai pada

modul hanya berisi :

```
case 3:
    System.out.println("Fitur ini akan kerjakan sebagai Tugas Mandiri.");
    break;
```

Pada pilihan 4/ case 4, program ngecek apakah sudah ada rekening aktif. Jika ada, program manggil method cekInformasi() untuk menampilkan informasi rekening.

Setiap case menggunakan break agar setelah proses selesai program keluar dari switch dan kembali menampilkan pilihan menu.

Pada case 0, program digunakan untuk pilihan keluar dari sistem. Nilai isRunning diubah dari true ke false, sehingga kondisi while salah dan perulangan berhenti. Setelah itu program menampilkan pesan bahwa sistem telah ditutup. Sementara itu, default digunakan apabila user memasukkan pilihan yang tidak tersedia sehingga program menampilkan pesan "Pilihan tidak Valid!!". Terakhir, input.close() digunakan untuk menutup Scanner setelah program selesai dijalankan.

2.2 Tugas Praktikum

2.2.1 Deskripsi Tugas

Untuk menguji pemahaman dalam melakukan manipulasi state (saldo) dan pengelolaan banyak objek secara dinamis, selesaikan tugas berikut:

- 1) Implementasi Fitur Tarik Tunai

- Tambahkan method tarikTunai(double nominal) ke dalam class Rekening.
- Aturan : Saldo tidak boleh menjadi minus dan minimal nominal Tarik tunai adalah 10.000. Jika pengguna mencoba menarik nominal yang lebih besar dari saldo saat ini atau nominal kecil dari 10.000, program tidak boleh crash/error, melainkan menampilkan peringatan ke konsol: "Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp[nominal]" atau "Transaksi Gagal : Minimal nominal penarikan 10.000".
- Terapkan method ini dengan menu nomor 3 pada class Main.

2) Tantangan Multi-Akun

Saat ini, aplikasi Main hanya dapat menyimpan 1 akun (Rekening akunAktif = null;). Jika pengguna memilih menu "Buka Rekening Baru" untuk kedua kalinya, akun lama akan tertimpa.

- Ubah arsitektur penyimpanan di Main.java menggunakan struktur data Collection. (Gunakan ArrayList<Rekening>).
- Ubah logika menu "Buka Rekening": Setiap kali pengguna membuka rekening baru, objek tersebut ditambahkan ke dalam ArrayList.
- (Bonus): Tambahkan menu baru untuk "Ganti Akun", di mana sistem akan meminta nomor rekening, kemudian mencari objek yang cocok di dalam ArrayList untuk dijadikan akunAktif

2.2.2 Pengerjaan Latihan/Tugas No. 1

- 1) Pada class Rekening saya menambahkan method tarikTunai(double nominal) seperti berikut.

```

public void tarikTunai(double nominal) {
    if (nominal < 10000) {
        System.out.println("Transaksi Gagal : Minimal nominal penarikan 10.000!");
    } else if (nominal > saldo) {
        System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp"
            + saldo);
    } else {
        saldo -= nominal;
        NumberFormat rupiah = NumberFormat.getCurrencyInstance(new Locale("id", "ID"));
        System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini : "
            + rupiah.format(saldo));
    }
}

```

Method ini untuk mengimplementasikan fitur penarikan uang seperti pilihan No. 3 pada menu utama. Seperti terlihat pada kodingan tsb, method menerima parametel nominal yang merupakan jumlah uang yang ingin ditarik oleh user. Di dalam method terdapat if else if untuk memastikan nominal penarikan sesuai perintah soal tugas. Jika nominal penarikan kurang atau sama dengan Rp10.000, program akan menampilkan pesan bahwa minimal nominal penarikan adalah Rp10.000. Selanjutnya, program memeriksa apakah nominal yang akan ditarik lebih besar dari saldo yang tersedia. Jika nominal penarikan melebihi saldo, program tidak akan melakukan transaksi dan menampilkan pesan “Transaksi Gagal: Saldo tidak mencukupi” beserta saldo yang dimiliki user.

Dengan begitu, saldo tidak akan menjadi minus dan program tetap berjalan tanpa mengalami error.

Jika nominal penarikan sudah memenuhi batas minimal dan tidak melebihi saldo, maka proses penarikan dapat dilakukan. Saldo akan dikurang dengan nominal penarikan ($\text{saldo} -= \text{nominal}$).

Setelah itu, program membuat format mata uang menggunakan NumberFormat dengan Locale("id", "ID") agar nominal ditampilkan dalam format Rupiah (seperti langkah kerja praktikum langkah 4, sebagai penugasan bonus). Kemudian program menampilkan pesan bahwa transaksi tarik tunai berhasil beserta nominal yang ditarik dan saldo terbaru.

- 2) Lalu terapkan method tarikTunai dengan pilihan menu No. 3/ case 3 pada class Main. Seperti berikut.

```
case 3 :
    if (akunAktif == null) {
        System.out.println("Error : Mohon maaf, anda belum memiliki norek!");
    }else {
        System.out.print("Masukkan nominal yang akan ditarik: ");
        double tarik = input.nextDouble();
        akunAktif.tarikTunai(tarik); //memanggil behavior/method
    }
    break;
```

Jika user memilih menu 3, program terlebih dahulu mengecek apakah akunAktif=null. Jika akunAktif null, berarti user belum memiliki rekening yang aktif sehingga program menampilkan pesan error. Jika sudah ada rekening aktif, user diminta memasukkan nominal yang ingin ditarik melalui input.nextDouble(), kemudian nominal tersebut disimpan dalam variabel tarik. Setelah itu, method tarikTunai() pada class rekening dipanggil menggunakan akunAktif.tarikTunai(tarik).

2.2.3 Pengerjaan Latihan/Tugas No. 2

- 1) Sesuai perintah tugas pada class main arsitektur penyimpanannya diubah menjadi menggunakan struktur data collection (ArrayList<Rekening>).

```
Package Pekan1;
import java.util.Scanner;
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        ArrayList<Rekening> bukaRekening = new
        ArrayList<>();
        Rekening akunAktif= null; //objek belum
        diinisialisasi (null)
        boolean isRunning = true;
```

Dengan ArrayList, setiap rekening yang dibuat tidak hanya menjadi akun aktif, tetapi juga disimpan ke dalam daftar menggunakan bukaRekening.add(akunAktif);. Jadi, jika user membuat beberapa rekening, seluruh objek rekening akan tetap tersimpan di dalam ArrayList, sedangkan akunAktif digunakan untuk menentukan rekening mana yang sedang digunakan.

- 2) Lalu pada baris pilihan menu, ditambahkan fitur/pilihan Ganti Akun (pilihan ke 5).

```
while (isRunning) {
    System.out.println("\nMenu Utama:");
    System.out.println("1. Buka Rekening Baru");
    System.out.println("2. Setor Tunai");
    System.out.println("3. Tarik Tunai");
    System.out.println("4. Cek Informasi Rekening");
    System.out.println("0. Keluar");
    System.out.println("5. Ganti Akun");
    System.out.println("Pilih Menu : ");

    int pilihan = input.nextInt();
    input.nextLine(); //membersihkan buffer enter
}
```

- 3) Selanjutnya pada case 1 bagian Buka Rekening, ditambahkan `bukaRekening.add(akunAktif);`. Disini `add()` berfungsi memasukkan objek rekening tersebut ke dalam `ArrayList`. Jadi, setiap kali user buka rekening baru, objek rekening akan ditambahkan ke daftar tanpa menghapus rekening sebelumnya.

```
case 1 :
    System.out.print("Masukkan No Rekening : ");
    String no = input.nextLine();
    System.out.print("Masukkan Nama Pemilik : ");
    String nama = input.nextLine();
    System.out.print("Masukkan Saldo Awal : ");
    double saldo = input.nextDouble();
    input.nextLine();

    //Instansiasi Object/Menjalankan Constructor
    akunAktif = new Rekening(no, nama, saldo);
    bukaRekening.add(akunAktif);
    break;
```

- 4) Setelah itu perubahan dilakukan dengan penambahan situasi case untuk pilihan 5 yang sudah ditambah sebelumnya. Yaitu pada case 5 ini program akan terlebih dahulu mengecek apakah `ArrayList` kosong menggunakan `bukaRekening.isEmpty()`. Jika belum ada rekening, program menampilkan pesan bahwa belum ada rekening yang tersedia. Jika sudah ada, user nanti akan diminta memasukkan nomor rekening yang ingin digunakan dan nomor tersebut disimpan dalam `noCari`.

Selanjutnya program memakai perulangan `for` (`Rekening rekening : bukaRekening`) untuk memeriksa setiap objek rekening yang tersimpan di dalam `ArrayList`. Nomor rekening dibandingkan menggunakan `rekening.nomorRekening.equals(noCari)`.

Jika nomor ada yang sesuai, objek tersebut disimpan ke rekeningDitemukan, kemudian akunAktif diubah menjadi rekening tersebut melalui $\text{akunAktif} = \text{rekeningDitemukan}$; Dengan begitu, user dapat berpindah dari satu rekening ke rekening lainnya tanpa membuat rekening baru sesuai dengan permintaan soal ke2 latihan/tugas.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa konsep class, object, constructor, dan method dapat diterapkan untuk membuat sistem perbankan sederhana menggunakan Java.

Pada praktikum ini, class Rekening digunakan untuk menyimpan data dan mengatur proses rekening seperti setor tunai, tarik tunai, dan pengecekan informasi. Penggunaan constructor memudahkan proses pembuatan objek rekening, sedangkan method digunakan untuk menjalankan berbagai operasi pada rekening.

Selain itu, penggunaan `ArrayList<Rekening>` memungkinkan program menyimpan lebih dari satu rekening dan fitur Ganti Akun memungkinkan user berpindah ke rekening yang berbeda. Dengan praktikum ini, dapat dipahami bagaimana objek dapat memiliki data (state) dan perilaku (behavior) yang dapat dimanipulasi melalui program.

DAFTAR PUSTAKA

- [1] Nurfiah, S.ST., M.Kom., *Modul Praktikum 1: Class, Object, Constructor dan Method — Interaksi Objek & Manipulasi State (Mini Bank)*.
- [2] Oracle, “Class NumberFormat,” *Java Platform, Standard Edition 8 API Specification*. [Java SE 8 NumberFormat API Documentation](#)